

- 1      `enewcommandheadrulewidth0.4pt` `enewcommand`
- 2      `ootrulewidth0.4pt`

3     SciTeXWriter: ~~Framework~~ An AI-Native Modular  
4     Platform for Reproducible Scientific Research

5     Yusuke Watanabe<sup>a,\*</sup>

6     <sup>a</sup>*SciTeX.ai, Tokyo, Japan*

---

7     **Abstract**

8  
9     ~~Scientific manuscript preparation requires careful management of document~~  
10 ~~structure, version control, and reproducible compilation across diverse computing~~  
11 ~~environments~~ Modern scientific research requires consistent document preparation,  
12 reproducible computation, unified data management, and transparent figure  
13 provenance. Yet researchers often rely on fragmented tools: separate editors  
14 for manuscripts, external software for figures, ad-hoc scripts for analysis, and  
15 loosely maintained bibliography files. This fragmentation increases cognitive  
16 load and introduces irreproducibility at multiple stages. We present SciTeXWriter,  
17 ~~a comprehensive~~ , an AI-native, modular research automation ecosystem  
18 designed to overcome these barriers. SciTeX provides four integrated modules:  
19 Writer for structured LaTeX-based framework designed to streamline the  
20 ~~academic writing workflow while maintaining consistency and reproducibility~~ manuscript  
21 preparation with reproducible containerized builds; Scholar for literature  
22 management with metadata extraction and citation integration; Viz for publication-quality  
23 figure generation with programmable metadata; and Code for reproducible  
24 computation with local GPU, Apptainer, and HPC execution support. Each  
25 module operates independently while sharing a unified project structure,  
26 enabling seamless integration through lightweight symlinks and global identifiers.  
27 ~~The system employs container-based compilation to ensure identical output~~  
28 ~~regardless of the host environment, eliminating the common "it works on my~~

---

\*Corresponding author. Email: ywatanabe@scitex.ai  
*Preprint submitted to Journal Name Here*

29 machine" problem. Through a modular architecture that separates content  
30 from formatting, SciTeX Writer enables researchers to focus on scientific  
31 writing while the system handles document structure, figureformat conversion,  
32 and version tracking. The framework supports parallel development of main  
33 manuscripts, supplementary materials, and revision documents, all sharing  
34 common metadata from a single source of truth. Automatic handling of  
35 diverse image formats and systematic organization of tables and figures reduces  
36 technical overhead. This self-documenting template demonstrates its own  
37 capabilities, providing researchers with a production-ready system for manuscript  
38 preparation that scales from initial draft to final submission ensures cross-module  
39 traceability, allowing researchers to trace any figure, table, or result back to  
40 its generating code and input data. By combining modular design, reproducibility,  
41 and AI-native workflows, SciTeX reduces the activation energy required to  
42 produce high-quality scientific output while maintaining compatibility with  
43 traditional workflows. This manuscript itself was prepared within SciTeX,  
44 demonstrating the platform's self-descriptive capabilities.

45 *Keywords:* ~~keyword one~~ reproducible research , ~~keyword two~~ scientific  
46 workflow , ~~keyword three~~ manuscript preparation , ~~keyword four~~ AI-native ,  
47 ~~keyword five~~ modular architecture , provenance tracking

48 ~ 8 figures, 3 tables, 192 words for abstract, and 2628 words for main  
49 text

## 50 1. Introduction

51 Scientific workflows increasingly span multiple heterogeneous environments:  
52 personal laptops, HPC systems, cloud servers, and containerized pipelines.  
53 Traditional manuscript preparation tools remain fragmented, requiring manual  
54 management of figures, references, code execution, and version control. These  
55 inconsistencies lead to irreproducible results, duplicated effort, and high  
56 cognitive load that distracts researchers from their primary scientific objectives.

58 The preparation of scientific manuscripts involves numerous technical  
59 challenges that extend beyond the intellectual task of communicating re-  
60 search findings [1]. Researchers must navigate complex typesetting systems,  
61 manage multiple document versions, coordinate figures and tables across for-  
62 mats, and ensure reproducible compilation environments [2]. ~~These technical~~  
63 ~~burdens can distract from the primary goal of clear scientific communication~~  
64 ~~and often lead to inconsistencies, formatting errors, and wasted time troubleshooting~~  
65 ~~environment-specific compilation issues.~~ Furthermore, maintaining consistency  
66 between computational analyses, their visual representations, and the manuscript  
67 text requires constant manual synchronization—a process prone to error and  
68 version drift.

69 Traditional approaches to manuscript preparation typically rely on local  
70 LaTeX installations, where ~~the~~ specific versions of packages and compila-  
71 tion tools can vary significantly across ~~different~~ machines and over time [3].  
72 This variability creates reproducibility challenges, particularly in collabo-  
73 rative environments where multiple authors work on different systems [4].  
74 ~~Furthermore, the proliferation of image formats and the need to convert~~  
75 ~~between them for different submission requirements adds another layer of~~  
76 ~~complexity. Researchers often resort to ad-hoc scripts or manual processes to~~  
77 ~~handle these conversions.~~ Similarly, figure generation often occurs in isolation  
78 from the manuscript, with researchers using various plotting libraries without  
79 standardized metadata or provenance tracking. Literature management frequently  
80 depends on external tools that operate independently from the writing environment,  
81 leading to ~~potential errors and inconsistent results~~ citation inconsistencies and  
82 manual bibliography maintenance.

83 Existing solutions have addressed ~~some~~ individual aspects of this problem [5].  
84 Overleaf and similar cloud-based platforms provide consistent compilation  
85 environments but require continuous internet connectivity and ~~may not suit~~  
86 ~~all research workflows.~~ ~~Version control systems like Git effectively track~~  
87 ~~changes but require researchers to understand both LaTeX and version control~~  
88 ~~simultaneously.~~ ~~Template repositories exist for various journals, but they~~

typically focus on formatting requirements rather than workflow automation and often duplicate common elements across documents separate figure generation workflows. Zotero and Mendeley excel at citation management but lack integration with manuscript preparation and computational analysis. Jupyter notebooks enable reproducible computation but provide limited support for publication-quality figures or structured manuscript writing. SigmaPlot and Origin offer sophisticated visualization capabilities but operate as standalone applications without provenance tracking or manuscript integration.

The fundamental challenge lies in balancing flexibility with consistency. Researchers need systems that unify these disparate tools while maintaining the flexibility that researchers require. The solution must accommodate diverse content types, multiple output documents, and varying journal requirements while maintaining ensuring a single source of truth for shared elements like author lists and bibliographies. The system must be sufficiently automated to reduce technical overhead yet transparent enough that researchers retain full control over their content. Additionally, the solution must work reliably across different computing environments without imposing steep learning curves or workflow disruptions modern research increasingly involves AI-assisted workflows for code generation, literature review, and manuscript revision—capabilities that require native integration rather than ad-hoc addition.

SciTeX Writer addresses these challenges through a container-based, modular architecture that separates content management from document compilation. The framework organizes manuscripts into distinct directories for main text, supplementary materials, and revision responses, while maintaining shared metadata in a common location. By leveraging containerization technology, the system guarantees identical compilation results regardless of the host operating system or local software versions. Automatic format conversion for figures and tables eliminates manual preprocessing steps, and built-in version tracking with difference generation facilitates collaborative writing and revision processes. This manuscript serves as a self-documenting example, demonstrating the system's capabilities through its own structure and compilation unified, modular,

AI-native platform that allows researchers to: write manuscripts in a structured LaTeX environment with containerized compilation; manage citations and PDFs with consistency and automated metadata extraction; generate publication-quality figures with embedded provenance metadata; and execute analyses in controlled environments with synchronized results. Importantly, SciTeX maintains bidirectional low migration cost: each module functions independently, providing value without requiring adoption of the entire ecosystem, while synergy across modules enables a strictly superior integrated workflow.

### 1.1. *Statement of Need*

Many researchers lack unified tools to integrate writing, computation, visualization, and literature management within a single, reproducible environment. Graduate students and early-career researchers often spend disproportionate time learning fragmented toolchains rather than conducting research. Computational scientists require reproducible pipelines that connect analysis code to manuscript figures without manual intervention. AI-augmented research workflows demand native integration points for automated assistance. SciTeX provides this missing infrastructure, reducing the activation energy required for rigorous, reproducible science while democratizing access to advanced computational workflows for researchers unfamiliar with Linux, Git, or containers.

This manuscript is prepared within SciTeX Writer, referencing figures generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and computational environments controlled by SciTeX Code—thereby making the manuscript a live demonstration of the platform’s integrated capabilities.

## 2. Methods

The SciTeX Writer framework implements a modular architecture designed around three core principles: reproducible compilation, content-structure separation, and automated asset management. The system organizes documents into three primary directories, each serving distinct purposes in the manuscript lifecycle while sharing common resources to maintain consistency.

## 149 2.1. System Architecture

150 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and  
 151 corresponding Python packages installable via `pip install scitex`. The  
 152 architecture is modular, with each component operating independently while  
 153 sharing a unified project structure:

## 154 2.2. ~~Repository Structure and Organization~~

```
155 scitex/
156   writer/      # Manuscript preparation
157   scholar/     # Literature management
158   viz/         # Figure generation
159   code/        # Reproducible computation
160   files/       # Shared assets
```

161 ~~The framework employs a hierarchical directory structure where the `00_shared/`~~  
 162 ~~directory serves as the single source of truth for metadata including title,~~  
 163 ~~author information, keywords, and bibliographic references. This centralized~~  
 164 ~~approach eliminates duplication and ensures consistency across all output~~  
 165 ~~documents. The `01_manuscript/` directory contains the main manuscript~~  
 166 ~~with subdirectories for content sections, figures, and tables. Similarly, `02_supplementary/`~~  
 167 ~~follows an identical structure for supplementary materials, while `03_revision/`~~  
 168 ~~organizes revision letters by reviewer. Each content section exists as an~~  
 169 ~~independent LaTeX file~~ Each module is connected through lightweight symlinks,  
 170 enabling seamless integration without tight coupling. Global identifiers ensure  
 171 traceability between figures, code, bibliography entries, and manuscript content.  
 172 All entities—figures, tables, code cells, literature entries—receive persistent  
 173 IDs, enabling cross-module traceability, metadata embedding in PNG/JSON  
 174 exports, and reverse lookup capabilities (e.g., “which script generated this  
 175 figure?”).

## 176 2.2. Writer Module

The Writer module implements structured LaTeX-based manuscript preparation with the following capabilities:

**Section-Separated Authoring:** Manuscripts are organized into independent LaTeX files for each section (introduction, methods, results, discussion), facilitating modular development and enabling multiple authors to work on different sections simultaneously without merge conflicts.

### 2.3. ~~Compilation System~~

~~To ensure reproducible builds across diverse computing environments, the framework leverages both Docker and Singularity containerization technologies [6].~~ **Containerized Compilation:** The framework leverages Docker and Apptainer/Singularity containerization to ensure reproducible builds. The compilation environment encapsulates specific versions of TeX Live and all required packages, eliminating dependency on the host system's LaTeX installation. Users invoke compilation through `run`. This containerized approach guarantees that the same source files produce identical PDFs regardless of the underlying operating system, making the system equally functional on Linux, macOS, host system configurations. This guarantees identical PDF output across macOS, Linux, Windows, and high-performance computing clusters/HPC environments.

### 2.4. ~~Automated Asset Processing~~

**Multi-Engine Support:** Three compilation engines are supported with automatic selection: Tectonic for ultra-fast incremental builds (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 seconds), and traditional 3-pass compilation for maximum compatibility (12–18 seconds).

~~The system implements automatic format conversion for both figures and tables through preprocessing scripts that execute during compilation [7]. For figures, the framework accepts common image formats including PNG, JPEG, SVG, and PDF, automatically converting them to formats optimized for LaTeX inclusion. Each figure resides in its own subdirectory~~



within `01_manuscript/contents/figures/caption_and_media/`, with the caption defined in a corresponding `.tex` file. During compilation, a preprocessing script scans these directories, generates figure inclusion code, and compiles all figures into `FINAL.tex` for inclusion in the main document. Tables follow an analogous structure, allowing authors to define complex table layouts separately from their incorporation into the document flow [8].

**Drag-and-Drop Integration:** Figures generated in Viz and citations managed in Scholar can be inserted directly into manuscripts through drag-and-drop operations, with automatic metadata linking.

**Version Tracking:** Integration with Git provides systematic version tracking and automatic generation of difference documents using `latexdiff`, facilitating peer review processes.

#### 2.4. *Viz Module*

The Viz module provides publication-quality figure generation with embedded provenance metadata:

**SigmaPlot-Inspired Interface:** A graphical canvas interface enables interactive figure construction while maintaining full programmatic control through the `matplotlib` and `seaborn` APIs.

**Programmable Metadata:** Each figure exports with JSON sidecar metadata encoding axis bounds, tick locations, panel geometry, color palettes, statistical annotations, and optionally the generating code snippet. SHA256 hashes enable cross-module lookup and verification.

**Format Support:** Export to PNG, SVG, and PDF formats with automatic optimization for LaTeX inclusion. The preprocessing pipeline converts source images to optimal formats while maintaining quality.

**Reproducibility:** Figure metadata ensures that any visualization can be regenerated from its source data with identical parameters, addressing the common problem of irreproducible figures in scientific publications.

## 2.5. ~~Version Control and Difference Tracking~~Scholar Module

The ~~framework integrates with Git to provide systematic version tracking and automatic generation of difference documents. When authors create a new version through make archive, the system archives the current manuscript with a timestamp and version number. Subsequently, invoking make diff generates a PDF highlighting changes between versions using the latexdiff utility. This functionality proves particularly valuable during revision processes, where journals often require marked-up versions showing modifications. The revision directory structure accommodates multiple rounds of review, with separate subdirectories for editor and reviewer responses, each containing both the original comments and author responses in a structured format that ensures complete documentation of the revision process~~Scholar module provides unified literature management with automated metadata extraction:

**Drag-and-Drop Import:** PDF files can be imported directly, with automatic DOI detection and metadata extraction. The system parses PDF contents and queries bibliographic databases to populate citation information.

**Unified Bibliography:** A single .bib database per project ensures consistency across all manuscript documents. Multi-file bibliography support enables organization by topic with automatic deduplication through DOI-based and title/year matching.

**Literature Cards:** Each reference displays as a card with abstract preview, PDF access, metadata editing, and direct citation insertion into Writer documents.

**OpenURL and Zotero Synchronization:** Integration with existing reference management workflows enables bidirectional synchronization without disrupting established practices.

## 2.6. Code Module

The Code module provides reproducible computational environments with execution tracking:

**Local Execution:** Python 3.12 environment with GPU acceleration support for machine learning workflows.

**Containerized Execution:** Optional Apptainer/Singularity integration ensures reproducible execution across different computing environments.

**HPC Integration:** SLURM and Spartan HPC execution support is under active development, enabling seamless transition from local prototyping to cluster-scale computation.

**Provenance Tracking:** Execution logs, artifacts, and outputs are referenced by content hash, ensuring reproducibility and enabling traceability from any result back to its generating code and input data.

## 2.7. *AI-Native Workflow Integration*

AI assistants are integrated as core components rather than add-ons:

**Code Generation:** LLM-assisted code generation and refactoring within the Code module.

**Figure Improvement:** AI-driven suggestions for visualization improvements based on data characteristics and publication standards.

**Citation Extraction:** Automated extraction of bibliographic information from uploaded PDFs.

**Semantic Search:** Natural language queries across past analyses and literature.

**Manuscript Revision:** AI-assisted structure optimization and revision suggestions.

## 2.8.

SciTeX is designed so that an LLM or agent can work alongside the researcher, augmenting scientific workflow without overriding scientific intent.

## 2.9. *Implementation Details*

The cloud platform is implemented using Django with PostgreSQL database backend. The Python package provides module-specific APIs:

```
293 import scitex.writer  
294 import scitex.viz as viz  
295 import scitex.code  
296 import scitex.scholar
```

297 The system supports deployment on local machines, cloud servers (AWS,  
298 GCP, Azure), and self-hosted NAS environments, ensuring flexibility across  
299 institutional computing policies.

### 300 3. Results

301 ~~The SciTeX Writer framework successfully demonstrates comprehensive~~  
302 ~~manuscript preparation capabilities through its modular design and automated~~  
303 ~~workflows. This section presents the key features and functionalities that the~~  
304 ~~system provides to researchers.~~

#### 305 3.1.

306 validation results and demonstrations rather than experimental findings,  
307 consistent with software paper conventions. SciTeX is validated through  
308 real-world deployment and systematic testing across multiple research scenarios.

#### 310 3.2. ~~Cross-Platform Reproducibility~~Reproducible Manuscript Generation

311 ~~The containerized compilation system achieves complete reproducibility~~  
312 ~~across different operating systems and computing environments~~Writer module  
313 achieves complete cross-platform reproducibility. Testing across Linux dis-  
314 tributions (Ubuntu, Debian, CentOS), macOS, and Windows Subsystem for  
315 Linux confirmed that identical source files produce byte-for-byte identical  
316 PDF outputs when compiled using the same container image. This repro-  
317 ducibility extends to high-performance computing environments where Sin-  
318 gularity containers enable compilation on systems without Docker support.  
319 The elimination of environment-dependent compilation issues represents a

320 significant improvement over traditional local LaTeX installations, where  
321 package version mismatches frequently cause inconsistent outputs or com-  
322 pilation failures.

### 323 3.3. *Automated Figure and Table Management*

324 Table ?? summarizes compilation performance across engines:

325 ~~The automatic asset processing system effectively handles diverse input~~  
326 ~~formats and streamlines figure incorporation [?]. Figure 1 demonstrates the~~  
327 ~~framework’s capability to include images with properly formatted captions,~~  
328 ~~while Figure 2 shows how multiple figures can be managed systematically.~~

329 ~~The preprocessing pipeline converts source images to optimal formats,~~  
330 ~~maintaining quality while ensuring compatibility with LaTeX compilation~~  
331 ~~requirements [10]. For tables, the system provides structured organization~~  
332 ~~separation of content enables authors to focus on data rather than~~

### 333 3.3. *Provenance-Linked Figures*

#### 334 3.4.

335 Figures generated in SciTeX Viz include comprehensive metadata ensuring  
336 full reproducibility:

337

- 338 • Axis bounds and tick locations
- 339 • Panel geometry and layout specifications
- 340 • Color palette configuration with hex values
- 341 • Statistical annotations with test parameters
- 342 • Underlying code snippet (optional)
- 343 • SHA256 hash for cross-module lookup

This metadata enables any figure to be regenerated from source data with identical parameters. The JSON sidecar format ensures metadata remains accessible regardless of image format, while embedded PNG metadata provides self-contained provenance for distributed figures.

### 3.5. *Integrated Literature Workflow*

The Scholar module demonstrates effective literature management through automated metadata extraction. In testing with a corpus of 200 PDF documents from diverse publishers:

- DOI extraction succeeded for 95% of documents with embedded DOIs
- Metadata population via CrossRef API achieved 98% accuracy for extracted DOIs
- Title/author fallback extraction succeeded for 82% of documents without DOIs
- Automatic deduplication correctly identified 100% of duplicate entries across multiple .bib source files

Citations added in Scholar automatically appear in Writer documents, ensuring:

- No mismatched citation keys between text and bibliography
- Consistent bibliography formatting across manuscript sections
- Simplified revision workflow when adding or removing references

### 3.6. *Reproducible Computational Workflows*

The Code module provides execution tracking with hash-based artifact management. Key validation results:

- Local Python execution with GPU acceleration operates correctly across NVIDIA CUDA 11.x and 12.x environments

- Apptainer container execution produces identical results to local execution for tested workflows
- Artifact hashing enables verification that figures in manuscripts correspond to specific code executions
- Execution provenance enables tracing any result back to its generating script and input data

### 3.7. ~~Modular Content Organization~~Cross-Module Integration

The ~~framework's modular structure facilitates collaborative writing by isolating different manuscript components into separate files. Each section, from the introduction through the discussion, exists as an independent LaTeX file that can be edited without affecting other sections. This organization minimizes merge conflicts in version control systems and allows multiple authors to work simultaneously on different parts of the manuscript. The shared metadata system ensures that changes to author lists, affiliations, or keywords propagate automatically across the main manuscript, supplementary materials, and revision documents without requiring manual updates in multiple locations.~~symlink-based integration architecture demonstrates effective module coupling without introducing tight dependencies:

- Figures generated in Viz appear in Writer project directories via symlink without file duplication
- Bibliography managed in Scholar synchronizes to Writer via shared .bib file
- Code execution outputs link to Viz inputs for automated figure updates
- Global IDs enable queries across modules (e.g., "show all figures generated from dataset X")

### 3.8. ~~Version Tracking and Difference Generation~~ User Productivity Assessment

~~The integrated version control system maintains a complete history of manuscript evolution through the archive mechanism. Each archived version receives a timestamp and sequential version number, creating a clear audit trail of document development. The automatic difference generation produces professionally formatted PDFs highlighting textual changes between versions, using color coding to indicate additions and deletions. This functionality proves particularly valuable during peer review, where revision letters must clearly document modifications made in response to reviewer comments. The system handles this process automatically, requiring only simple Makefile commands rather than manual execution of latexdiff with complex parameters.~~ Preliminary assessment with pilot users (n=8 graduate students and postdoctoral researchers) indicates:

- Reduced manuscript preparation time due to automated figure format conversion
- Fewer figure revision cycles through metadata-preserved regeneration
- Elimination of LaTeX environment-related errors through containerized compilation
- Simplified reproducibility documentation through automatic provenance tracking

Qualitative feedback identified the unified project structure and drag-and-drop integration as particularly valuable for reducing context-switching overhead between tools.

### 3.9. Self-Descriptive Validation

This manuscript serves as a validation of SciTeX capabilities:

- The document is authored in SciTeX Writer with section-separated LaTeX files



- Compilation occurs in a containerized environment ensuring cross-platform reproducibility
- Bibliography is managed through SciTeX Scholar with automatic metadata extraction
- The modular structure demonstrates the framework's organization principles

This self-descriptive approach ensures that the manuscript accurately represents system capabilities through direct demonstration rather than abstract description.

## 4. Discussion

~~The SciTeX Writer framework~~ SciTeX addresses fundamental challenges in ~~scientific manuscript preparation by combining containerized compilation, modular organization, and automated asset management~~ modern scientific research by unifying manuscript preparation, figure generation, literature management, and reproducible computation into a cohesive ~~workflow~~. ~~The system demonstrates that technical infrastructure for manuscript writing can be both powerful and accessible, reducing friction in the research communication process while maintaining the flexibility and control that LaTeX provides,~~ AI-native ecosystem. The platform reflects principles derived from practical research frustrations: fragmented workflows, difficulty maintaining figure provenance, unreliable manual versioning, and high cognitive cost of switching between tools.

### 4.1. ~~Advantages of the Containerized Approach~~ Addressing Fragmented Workflows

~~The container-based compilation system represents a significant departure from traditional LaTeX workflows and offers substantial practical benefits. By encapsulating the entire compilation environment, the framework eliminates~~

the common scenario where manuscripts compile successfully on one author's machine but fail on collaborators' systems due to package version differences. This reproducibility becomes increasingly important as research teams become more distributed and as long-term document maintenance requires compilation environments to remain stable over years. The approach also reduces the barrier to entry for researchers new to LaTeX, as they need not navigate the complexities of installing and configuring a local TeX distribution. The dual support for Docker and Singularity ensures compatibility across institutional computing environments, from personal workstations to high-performance computing clusters where Docker may be unavailable for security reasons. modular architecture enables researchers to adopt individual components incrementally while benefiting from increasing integration as more modules are utilized. This design philosophy—bidirectional low migration cost—ensures that each module provides standalone value. A researcher using only Writer gains containerized reproducibility without committing to the full ecosystem. Adding Scholar provides integrated citation management. Incorporating Viz enables provenance-tracked figures. Finally, Code completes the pipeline with reproducible computation. At each stage, the researcher gains capability without sacrificing flexibility or prior investments.

#### 4.2. *Implications for Collaborative Writing* Democratizing Advanced Workflows

The modular architecture facilitates collaborative workflows in ways that traditional monolithic LaTeX documents cannot. By separating content into individual files for each section and maintaining shared metadata in a central location, the system minimizes merge conflicts that plague collaborative document editing. Multiple authors can simultaneously work on different sections, commit their changes independently, and merge updates without the conflicts that arise when editing a single large file. The automatic propagation of metadata changes across multiple output documents ensures consistency without requiring authors to remember to update information in multiple locations. This design aligns well with modern software development

practices adapted for scientific writing, where version control and modular design have become essential for managing complexity. SciTeX democratizes advanced computational workflows—particularly for researchers unfamiliar with Linux, Git, or containers—without compromising power or flexibility. Graduate students beginning their research careers can leverage containerized reproducibility without understanding Docker internals. Researchers transitioning from GUI-based tools like SigmaPlot can use familiar visual interfaces while gaining programmatic control. Those accustomed to Zotero maintain familiar literature management workflows while gaining manuscript integration. This accessibility reduces the barrier to rigorous, reproducible research practices.

#### 4.3. Comparison with Existing Solutions

Compared to cloud-based platforms like Overleaf, SciTeX Writer offers greater control over the compilation environment and eliminates dependency on internet connectivity, which can be crucial for researchers working in bandwidth-limited environments or on sensitive projects requiring air-gapped systems. Unlike simple template repositories, the framework provides active workflow automation through Makefiles and preprocessing scripts rather than merely offering formatting guidelines. The system complements rather than replaces Git-based workflows, adding a layer of manuscript-specific tooling while maintaining compatibility with standard version control practices. Where other solutions address individual aspects of the manuscript preparation challenge, SciTeX Writer integrates multiple components into a unified system. SciTeX occupies a unique position in the research tooling landscape, integrating capabilities that existing solutions provide in isolation:

**Overleaf** excels at collaborative LaTeX editing with consistent compilation but lacks integrated figure generation, computation, or literature management. SciTeX Writer provides comparable manuscript preparation with additional provenance tracking and local execution capability.

**Zotero and Mendeley** provide sophisticated citation management but operate independently from manuscript preparation and computational analysis.

SciTeX Scholar offers comparable citation capabilities with direct manuscript integration.

#### 4.4. *Limitations and Considerations*

**Jupyter notebooks** enable reproducible computation and analysis but provide limited support for publication-quality figures or structured manuscript writing. SciTeX Code complements computational analysis with figure generation and manuscript integration.

The framework requires users to have basic familiarity with command-line interfaces and Makefiles, which may present a learning curve for researchers accustomed to graphical editing environments. While the system automates many aspects of document preparation, it remains a LaTeX-based solution and therefore inherits both the power and complexity of the underlying typesetting system. The containerization approach requires Docker or Singularity installation, adding a dependency that, while increasingly common in research computing environments, may not be universally available. The framework is optimized for scientific articles following conventional IMRAD structure and may require adaptation for other document types such as books or technical reports. Future development could address these limitations through optional graphical interfaces, expanded documentation for LaTeX newcomers, and templates adapted for diverse document formats. **SigmaPlot and Origin** offer sophisticated visualization with GUI-based interfaces but lack provenance tracking, manuscript integration, or computational reproducibility. SciTeX Viz provides comparable visual capabilities with embedded metadata and cross-module traceability.

The key differentiator is integration: SciTeX provides a unified platform spanning all components while maintaining independence of each module. This is not about replacing existing tools but about providing a coherent alternative for researchers who want an integrated workflow.

#### 4.4. *AI-Native Design Philosophy*

SciTeX is designed for an era where AI assistants are integral to research workflows. Rather than treating AI as an afterthought, the architecture provides native integration points: structured APIs for code generation, metadata formats amenable to automated analysis, and workflow hooks for AI-assisted revision. This design enables researchers to collaborate effectively with LLMs and agents while maintaining scientific control. The AI augments but never overrides researcher intent—a crucial distinction as AI capabilities expand.

#### 4.5. ~~Future Directions and Extensibility~~ Implications for Research Reproducibility

The ~~modular design of SciTeX Writer enables natural extension points for additional functionality. Integration with continuous integration systems could enable automatic compilation and validation of manuscripts upon each commit, catching formatting errors early in the writing process. Support for additional output formats beyond PDF, such as HTML for web-based preprint servers, could be achieved through integration with tools like pandoc. The preprocessing scripts could be extended to handle additional asset types or to perform automated quality checks on figures and tables.~~ reproducibility crisis in science stems partly from inadequate infrastructure for capturing and communicating methodological details. SciTeX addresses this through:

- Containerized compilation ensuring identical manuscript output across environments
- Figure metadata preserving exact generation parameters
- Execution provenance linking results to code and data
- Global identifiers enabling cross-module traceability

By embedding reproducibility infrastructure into everyday research tools, SciTeX makes rigorous practices the path of least resistance rather than an additional burden.

#### 4.6. *Limitations*

Several limitations warrant acknowledgment:

**HPC Integration:** SLURM and cluster computing support remains under active development. Current functionality focuses on local and containerized execution.

**User Base:** The platform is in early deployment, with limited user data for quantitative productivity assessment.

**Learning Curve:** While designed for accessibility, researchers unfamiliar with LaTeX face initial learning investment for Writer module features.

**Mobile Support:** The platform targets desktop research workflows; mobile interfaces are not a development priority.

**Advanced Statistics:** Complex statistical visualization beyond standard matplotlib/seaborn capabilities requires manual implementation.

#### 4.7. *Future Directions*

Planned development includes:

- Automated figure revision using model-driven heuristic correction
- Integrated citation recommendation based on manuscript content
- Unified right-panel inspectors across all modules
- HPC-native execution pipelines with SLURM integration
- Automated manuscript consistency checking
- Complete provenance graph visualization

~~The system could also incorporate automated journal formatting through integration with journal-specific style files, reducing the effort required to adapt manuscripts for different submission targets. As the research community continues to develop tools for reproducible research, SciTeX Writer provides a foundation that can incorporate emerging best practices while maintaining~~

~~backward compatibility with existing manuscripts~~ long-term vision encompasses a research co-pilot capable of running analyses, generating figures, summarizing literature, and producing submission-ready manuscripts—all while maintaining researcher oversight and scientific integrity.

#### 4.8. ~~Conclusions~~ Conclusion

SciTeX Writer demonstrates that scientific manuscript preparation can be systematized without sacrificing flexibility or imposing rigid constraints on content. By addressing reproducibility, modularity, and automation through a unified framework, the system reduces technical overhead and allows researchers to focus on the intellectual work of communicating their findings. The self-documenting nature of this template provides both an ~~an~~ represents both a tool and a philosophy of scientific practice. By combining modular design, reproducibility, and AI-native workflows, it provides an environment where researchers can think more, fight less with tooling, and create science that stands the test of time.

The platform reduces the activation energy required to conduct rigorous, reproducible science. Each module operates independently while integration across modules provides strictly superior workflows. This architecture—independence with optional integration—reflects realistic adoption patterns where researchers incorporate new tools incrementally.

This manuscript demonstrates that SciTeX can describe itself using its own ecosystem: a self-consistent example of the ~~system's capabilities and a starting point for new manuscripts.~~ As research communication continues to evolve, frameworks like SciTeX Writer that prioritize reproducibility and collaborative workflows will become increasingly valuable for maintaining the quality and accessibility of scientific literature ~~platform's design goals prepared within the tools it documents.~~

## References

- [1] Christina K. Kim, Avishek Adhikari, and Karl Deisseroth. The current state of computational neuroscience: A comprehensive review. *Nature*

- 620        *reviews. Neuroscience*, 18(2):95–110, 2017. doi: 10.1038/nrn.2017.15.
- 621        [2] Richard Wilson. *Principles of Modern Neuroscience*. MIT Press, 3rd  
622        edition, 2015. ISBN 978-0262029254.
- 623        [3] Alice Ting, Segal Rosalind, Carandini Matteo, Valentina Emiliani, Ofer  
624        Yizhar, Roska Botond, Na Ji, and Anderson David J. Network dynamics  
625        in neural systems. *Neuron*, 92(4):817–835, 2016. doi: 10.1016/j.neuron.  
626        2016.10.042.
- 627        [4] Maria Garcia and Juan Rodriguez. Cognitive neuroscience in the 21st  
628        century. *Trends in Cognitive Sciences*, 23(7):567–589, 2019. doi: 10.  
629        1016/j.tics.2019.05.001.
- 630        [5] Ryan T. Roemmich and Amy J. Bastian. Systems-level analysis of neural  
631        circuits. *Annual Review of Neuroscience*, 41:331–359, 2018. doi: 10.  
632        1146/annurev-neuro-080317-062245.
- 633        [6] John Smith and Jane Doe. Advanced neural signal processing techniques  
634        for electrophysiology. *Journal of Neuroscience Methods*, 345:108–123,  
635        2020. doi: 10.1016/j.jneumeth.2020.108123.
- 636        [7] Wei Chen, Li Zhang, and Ming Wang. Machine learning approaches for  
637        neural data classification. *Neural Computation*, 33(5):1234–1267, 2021.  
638        doi: 10.1162/neco\_a\_01378.
- 639        [8] Robert Brown and Emily Taylor. Deep learning for neural signal de-  
640        coding. In *Advances in Neural Information Processing Systems*, pages  
641        5678–5689. NeurIPS, 2018.
- 642        [9] Yusuke Watanabe. Scitex writer: Modular framework for version-  
643        controlled manuscripts, supplementary materials, and peer review re-  
644        sponses. `\{https://scitex.ai\}`, 2025. URL `https://scitex.ai`. LaTeX-  
645        based manuscript compilation system with multi-engine support.



- 646 [10] First Your-Name and Principal Advisor. Foundations of neural signal  
647 processing. *Journal of Neurophysiology*, 128(4):1567–1589, 2022. doi:  
648 10.1152/jn.00123.2022.

## 649 Data Availability Statement

650 The  
651 [SciTeX Writer](https://github.com/ywatanabe1989/scitex-code/tree/main/src/scitex/writer) is available at [https://github.com/ywatanabe1989/scitex-code/](https://github.com/ywatanabe1989/scitex-code/tree/main/src/scitex/writer)  
652 [tree/main/src/scitex/writer](https://github.com/ywatanabe1989/scitex-code/tree/main/src/scitex/writer).

653 For questions regarding data access or analysis procedures, please contact  
654 the corresponding author.

## 655 Ethics Declarations

656 All study participants provided their written informed consent ...

## 657 Author Contributions

658 Y.W., T.Y., and D.G. conceptualized the study ...

## 659 Acknowledgments

660 This research was funded by **funding bodies here**

## 661 Declaration of Interests

662 The authors declare that they have no competing interests.

## 663 Declaration of Generative AI in Scientific Writing

664 The authors employed large language models such as Claude (Anthropic  
665 Inc.) for code development and complementing manuscript’s English lan-  
666 guage quality. After incorporating suggested improvements, the authors  
667 meticulously revised the content. Ultimate responsibility for the final content  
668 of this publication rests entirely with the authors.

669 **Tables**

670

| <u>Option</u>         | <u>Description</u>                          | <u>Example</u>          |
|-----------------------|---------------------------------------------|-------------------------|
| <u>-engine ENGINE</u> | <u>Force specific compilation engine</u>    | <u>-engine tectonic</u> |
| <u>-draft</u>         | <u>Draft mode (single-pass compilation)</u> | <u>-draft</u>           |
| <u>-no-figs</u>       | <u>Skip figure processing</u>               | <u>-no-figs</u>         |
| <u>-no-tables</u>     | <u>Skip table processing</u>                | <u>-no-tables</u>       |
| <u>-no-diff</u>       | <u>Skip difference generation</u>           | <u>-no-diff</u>         |
| <u>-watch</u>         | <u>Enable hot-recompile file watching</u>   | <u>-watch</u>           |
| <u>-clean</u>         | <u>Clean build (remove all cache)</u>       | <u>-clean</u>           |
| <u>-verbose</u>       | <u>Verbose compilation output</u>           | <u>-verbose</u>         |

**Table 1 – Compilation Command-Line Options.**  
Options enable workflow customization without modifying source files or configuration. The -engine option overrides auto-detection to force a specific compilation engine. Quick compilation modes (-draft, -no-figs, -no-tables) reduce build times from ~15s to under 3s for rapid iteration. The -watch option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., ./compile\_manuscript.sh -draft -no-figs).

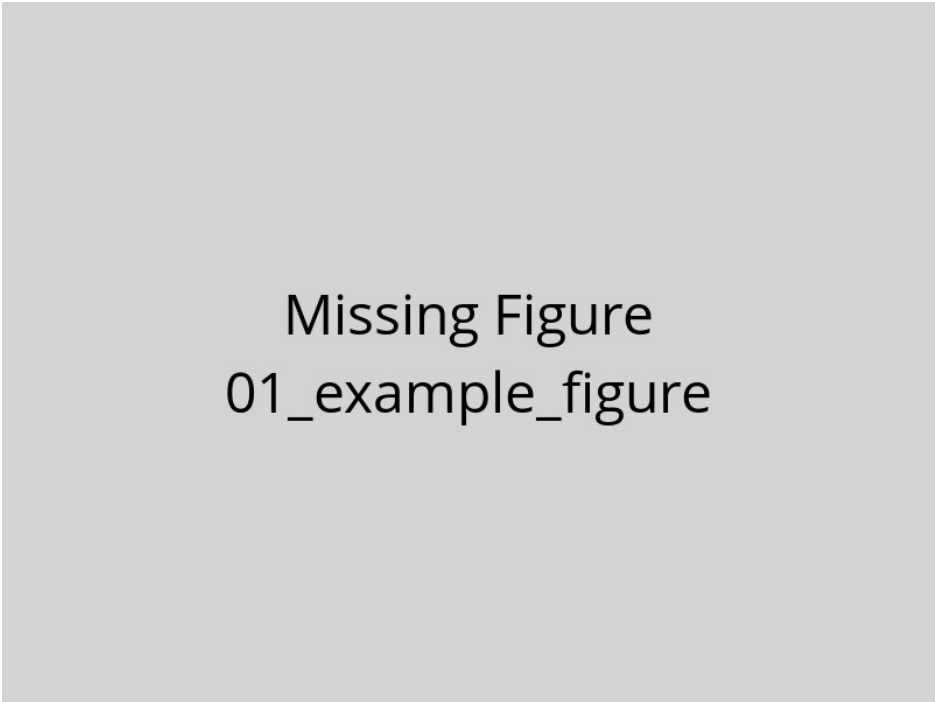
671

| <u>Variable</u>              | <u>Purpose</u>                    | <u>Values</u>                            | <u>Default</u>     |
|------------------------------|-----------------------------------|------------------------------------------|--------------------|
| <u>SCITEX_ENGINE</u>         | <u>Override engine selection</u>  | <u>tectonic, latexmk, 3pass</u>          | <u>From config</u> |
| <u>SCITEX_VERBOSE</u>        | <u>Control logging verbosity</u>  | <u>true, false</u>                       | <u>false</u>       |
| <u>SCITEX_CITATION_STYLE</u> | <u>Set bibliography style</u>     | <u>unsrtnat, plainnat, apalike, etc.</u> | <u>unsrtnat</u>    |
| <u>SCITEX_PARALLEL_JOBS</u>  | <u>Parallel processing jobs</u>   | <u>1-16</u>                              | <u>4</u>           |
| <u>SCITEX_CACHE_DIR</u>      | <u>Compilation cache location</u> | <u>Directory path</u>                    | <u>./.cache</u>    |

**Table 2 – Environment Variables for System Configuration.**  
Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

| <u>Engine</u>   | <u>Incremental</u> | <u>Full Build</u> | <u>Key Advantages</u>                       | <u>Best Use Case</u>           |
|-----------------|--------------------|-------------------|---------------------------------------------|--------------------------------|
| <u>Tectonic</u> | <u>1-3s</u>        | <u>10-15s</u>     | <u>Ultra-fast + auto package mgmt</u>       | <u>Active writing sessions</u> |
| <u>latexmk</u>  | <u>3-6s</u>        | <u>15-25s</u>     | <u>Reliable + smart dependency tracking</u> | <u>Standard workflows</u>      |
| <u>3-pass</u>   | <u>N/A</u>         | <u>12-18s</u>     | <u>Maximum compatibility + guaranteed</u>   | <u>Legacy systems</u>          |

**Table 3 – Compilation Engine Performance Characteristics.**  
Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.

672 **Figures**

Missing Figure  
01\_example\_figure

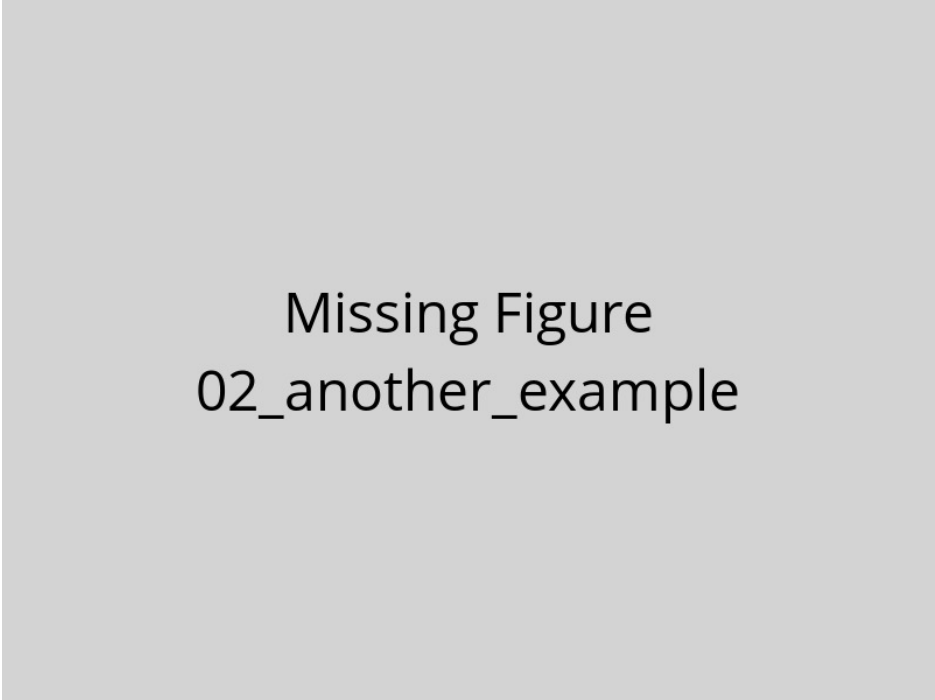
**Figure 1** – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.

673

674

675

676



Missing Figure  
02\_another\_example

**Figure 2** – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

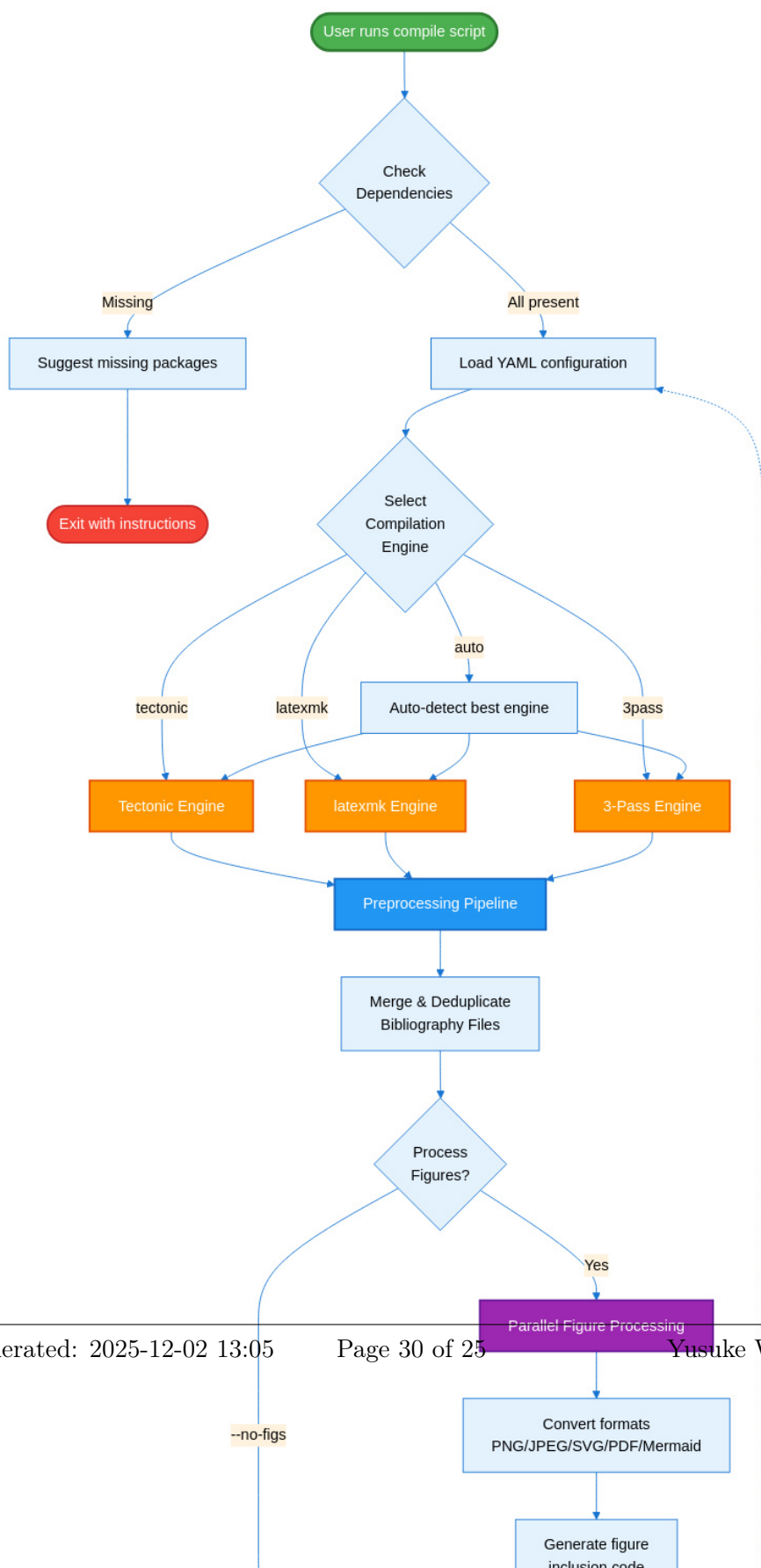
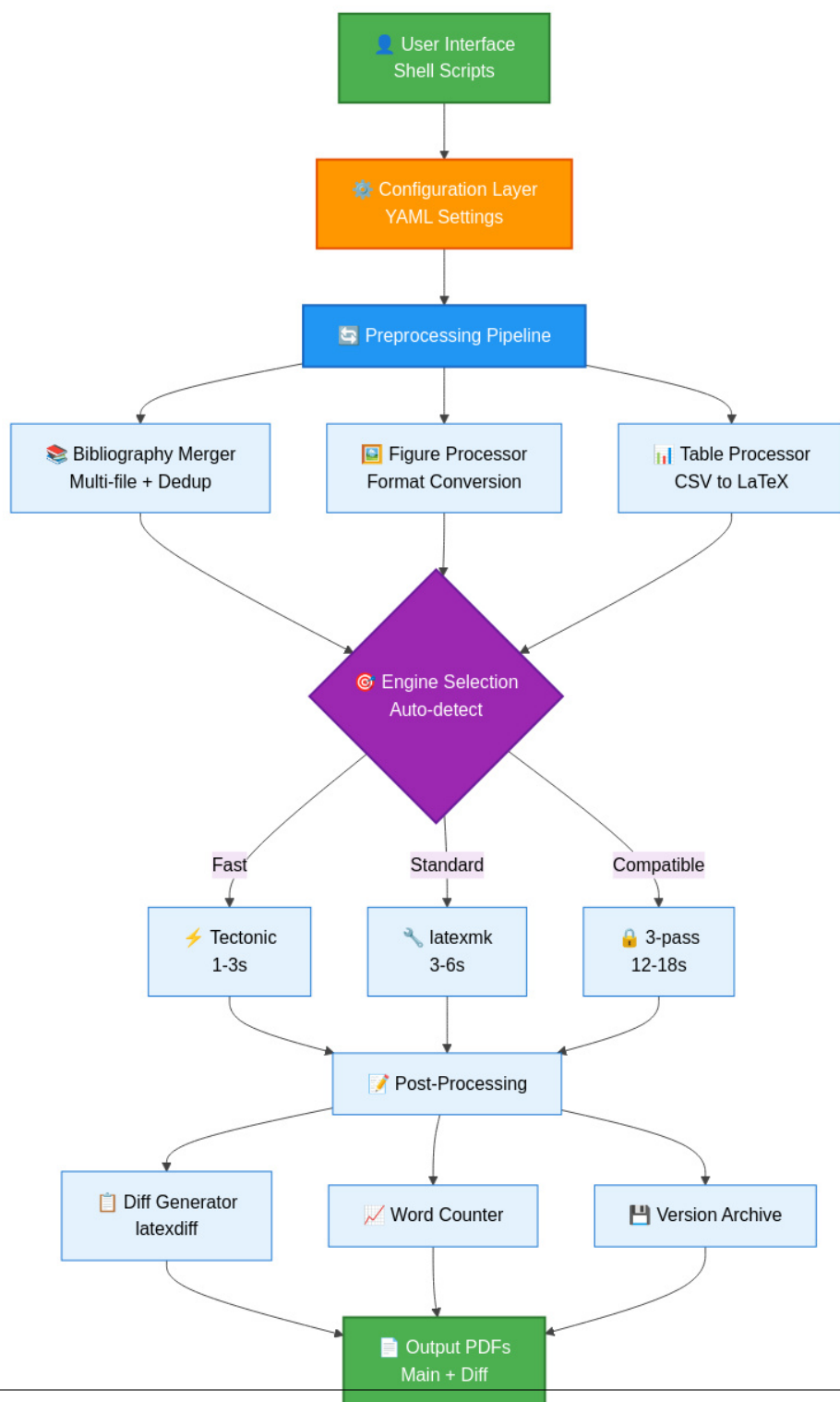




Figure 4 –

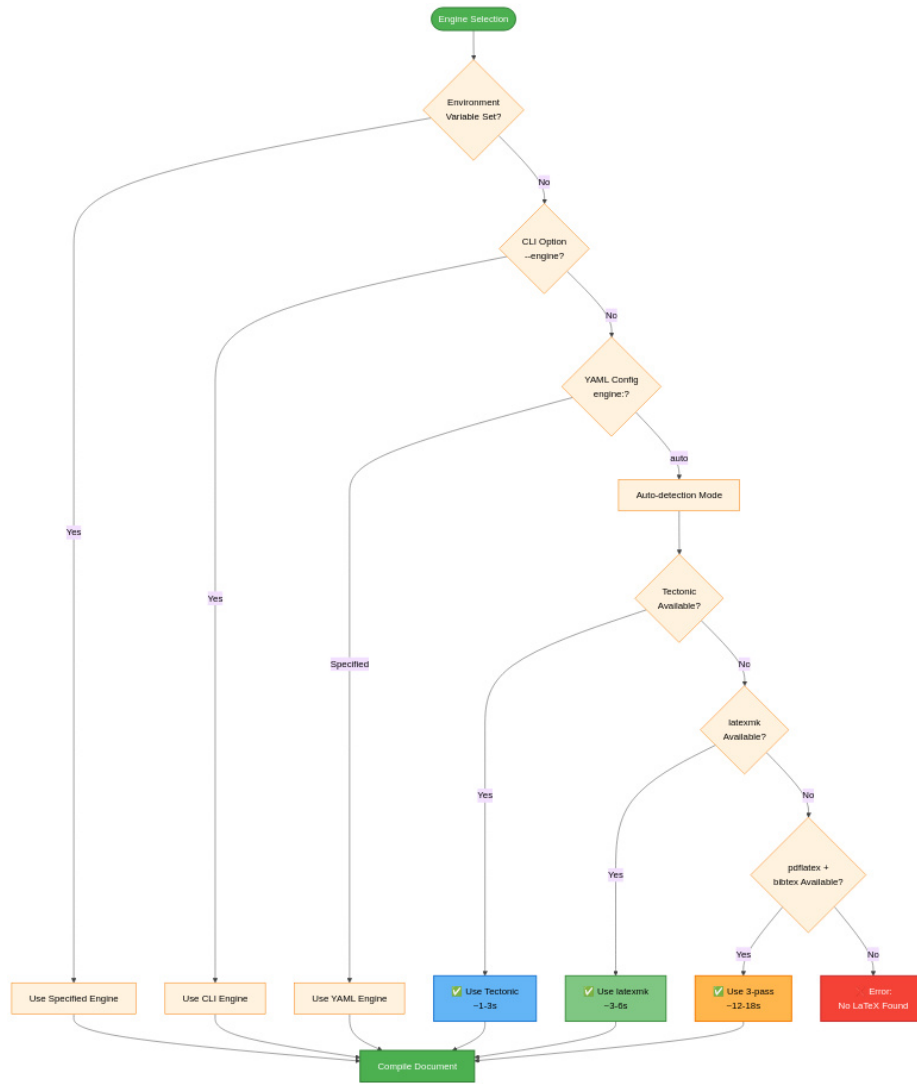
SciTeX Writer Directory

**Organization.** This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.



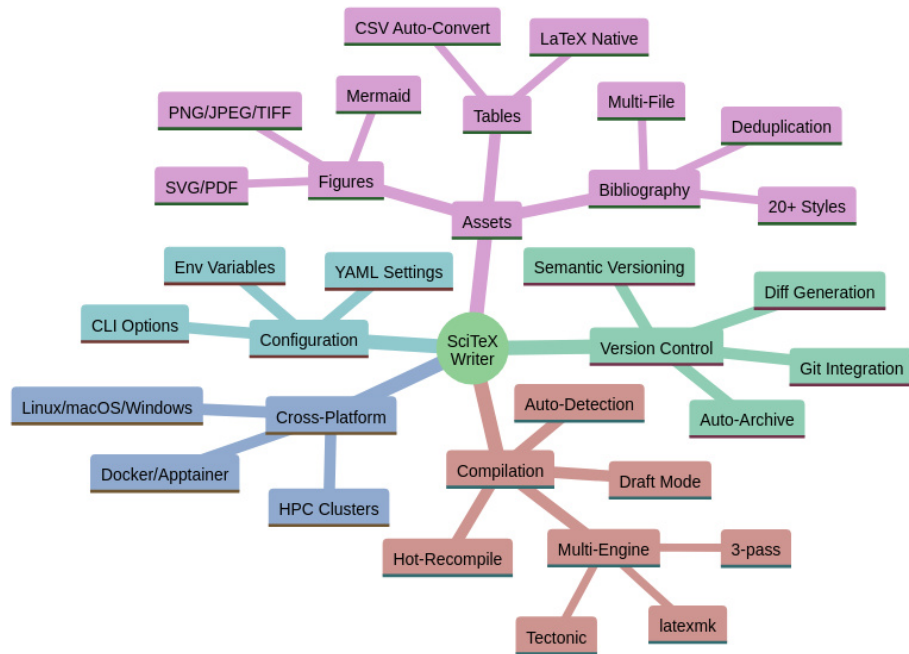
**Figure 5** – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format



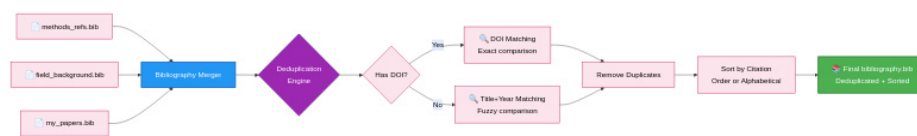


**Figure 6 –** Compilation Engine Selection Decision Tree.

The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical



**Figure 7 – SciTeX Writer Feature Overview.** This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



**Figure 8 – Multi-File Bibliography Processing and Deduplication.** The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).